

Unit-2

Inheritance and Polymorphism:-

1. Types of inheritance
2. Member access rules
3. Super uses
4. Using final with inheritance
5. The object class and its methods
6. Method Overriding
7. Dynamic Binding
8. Abstract Classes
9. Abstract Methods

Inheritance:-

Defination:- *When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.*

- We can achieve inheritance by using the **extends** keyword.
- Inheritance represents the **IS-A relationship** which is also known as a *parentchild* relationship.

Terms used in Inheritance

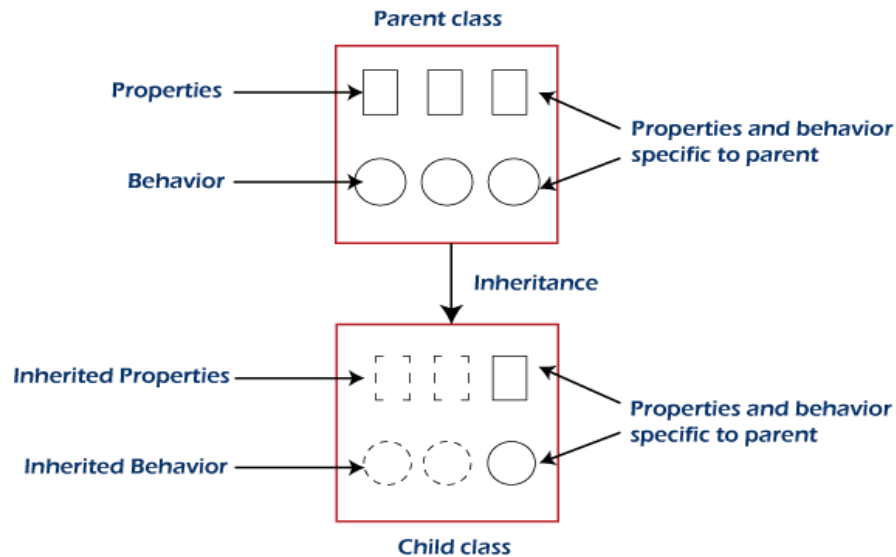
Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.

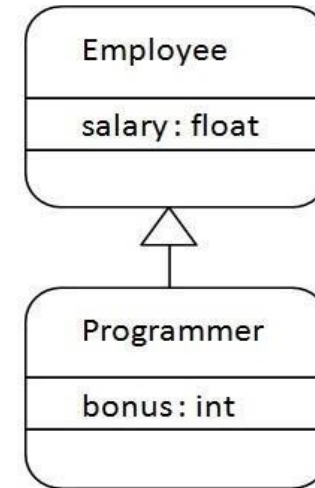
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
//methods and fields
}
```



Inheritance Example



Example program:-**class**Employee{

float salary=40000;

}

class Programmer **extends**Employee{ **int** bonus=10000; **publicstaticvoid**
main(String args[]){

Programmer p=**new**Programmer();

System.out.println("Programmer salary is:"+p.salary); System.out.println("Bonus of
Programmer is:"+p.bonus);

}

}

Output

:-

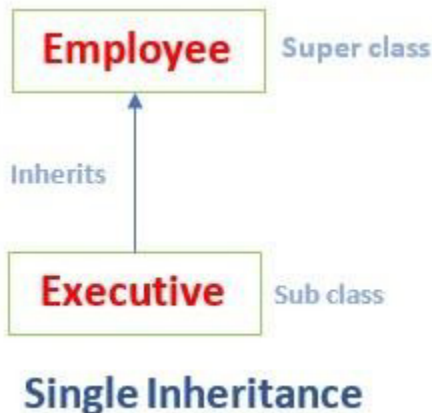
```
Programmer salary is:40000.0 Bonus of programmer is:10000
```

1.Types of Inheritance:-

- o Single Inheritance
- o Multi-level Inheritance
- o Hierarchical Inheritance
- o Hybrid Inheritance

1.Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes it is also known as si



Executive.java

class Employee

```
{  
float salary=34534*12;  
}
```

public class Executive **extends** Employee

```
{  
float bonus=3000*6;  
publicstaticvoidmain(String args[])
```

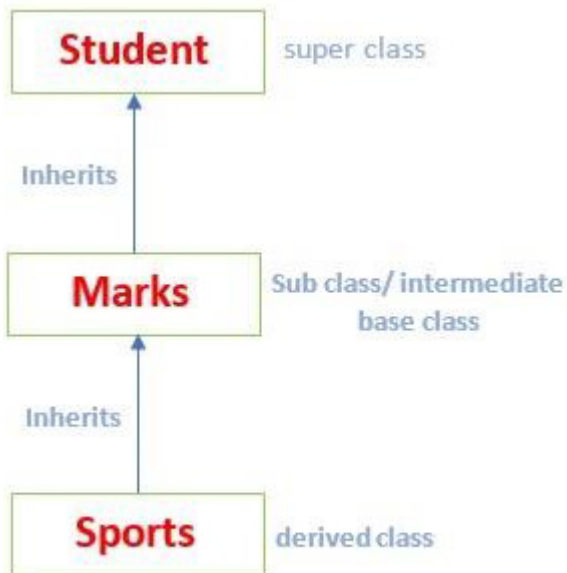
```
{  
Executive obj=newExecutive();  
System.out.println("Total salary credited: "+obj.salary); System.out.println("Bonus of six months: "+obj.  
bonus);  
}  
}
```

Output:

Total salary credited: 414408.0 Bonus of six months: 18000.0

2.Multi-level Inheritance

In **multi-level inheritance**, a class is derived from a class which is also derived from another class is called multi-level inheritance. In simple words, we can say that a class that has more than one parent class is called multi-level inheritance. Note that the classes must be at different levels. Hence, there exists a single base class and single derived class but multiple intermediate base classes.



Multi-level Inheritance

Multilevel InheritanceExample.java

//super class

class Student

{ **int** reg_no; **void** getNo(**int** no)

{

reg_no=no;

}

void putNo()

{

System.out.println("registration number= "+reg_no);

}

}

//intermediate sub class

class Marks **extends** Student

{

float marks;

void getMarks(**float** m)

{

marks=m;

}

void putMarks()

{

System.out.println("marks= "+marks);

}

}

//derived class

class Sports **extends** Marks

{

float score;

void getScore(**float**scr)

{

score=scr;

}

void putScore()

{

System.out.println("score= "+score);

}

}

public class MultilevelInheritanceExample

{

public static void main(String args[])

{

Sports ob=**new**Sports();

ob.getNo(0987);

ob.putNo();

ob.getMarks(78);

ob.putMarks();

ob.getScore(68.7);

ob.putScore();

}

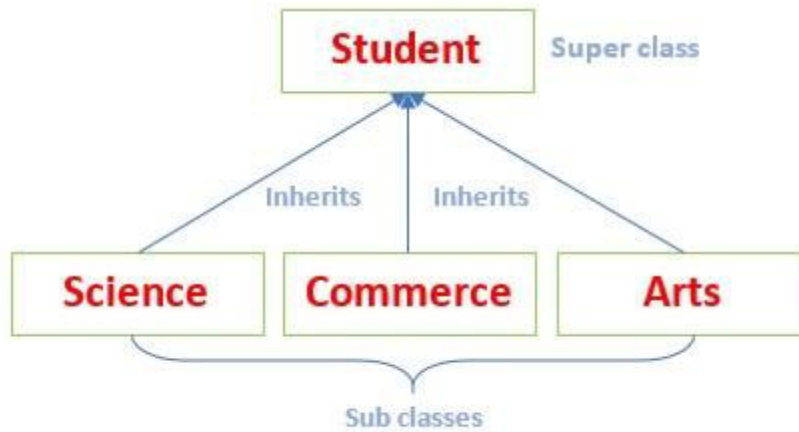
}

Output

```
registration number= 0987  
marks= 78.0  
score= 68.7
```

3.Hierarchical Inheritance

If a number of classes are derived from a single base class, it is called **hierarchical inheritance**.



Hierarchical Inheritance

HierarchicalInheritanceExample.java

```
//parent class
class Student
{
    publicvoidmethodStudent()
    {
        System.out.println("The method of the class Student invoked.");
    }
}

class Science extends Student
{
    publicvoidmethodScience()
    {
        System.out.println("The method of the class Science invoked.");
    }
}

class Commerce extends Student
{
    publicvoidmethodCommerce()
    {
        System.out.println("The method of the class Commerce invoked.
");
    }
}
```

```

class Arts extends Student
{
public void methodArts()
{
System.out.println("The method of the class Arts invoked.");
}
}

public class HierarchicalInheritanceExample
{
public static void main(String args[])
{
Science sci = new Science();
Commerce comm = new Commerce();
Arts art = new Arts();
//all the sub classes can access the method of super class
sci.methodStudent();

comm.
methodStudent();
art.
methodStudent();
}
}

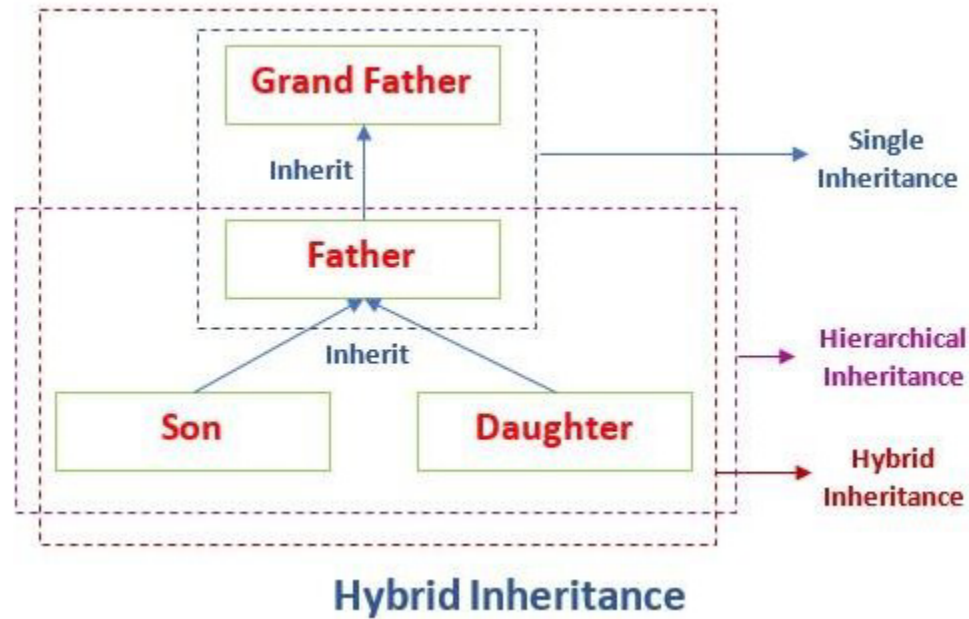
```

The method of the class Student invoked.

The method of the class Student invoked. The method of the class Student invoked.

4. Hybrid Inheritance

Hybrid means consist of more than one. Hybrid inheritance is the combination of two or more types of inheritance.



Let's implement the hybrid inheritance mechanism in a Java program.

Daughter.java

```
//parent class
class GrandFather
{
    public void show()
    {
        System.out.println("I am grandfather.");
    }
}

//inherits GrandFather properties
class Father extends GrandFather
{
    public void show()
    {
        System.out.println("I am father.");
    }
}

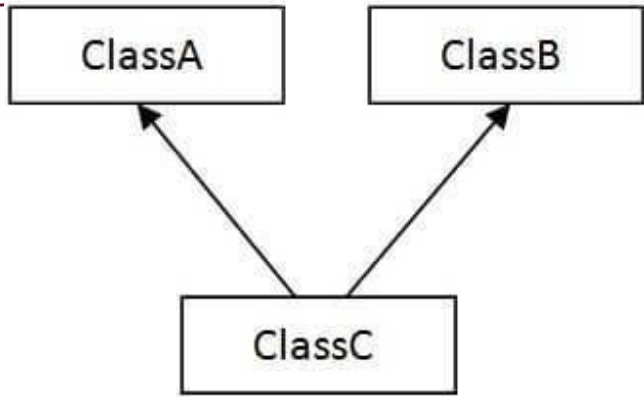
//inherits Father properties
class Son extends Father
{
    public void show()
    {
        System.out.println("I am son.");
    }
}
```

```
//inherits Father properties
public class Daughter extends Father
{
    public void show()
    {
        System.out.println("I am a daughter.");
    }
    public static void main(String args[])
    {
        Daughter obj = new Daughter();
        obj.show();
    }
}
```

Output:

```
I am daughter.
```

5. Multiple Inheritance (not supported)



4) Multiple

Java does not support multiple inheritances due to ambiguity. For example, consider the following Java program.

Demo.java

class Wishes

```
{  
void message()  
{  
System.out.println("Best of Luck!!");  
}  
}
```

```

class Birthday
{
void message()
{
System.out.println("Happy Birthday!!");
}
}

public class Demo extends Wishes, Birthday //considering a
scenario
{
public static void main(String args[])
{
Demo obj=new Demo();
//can't decide which classes' message() method will be invoked
obj.message();
}
}

```

- The above code gives error because the compiler cannot decide which message() method is to be invoked. Due to this reason, Java does not support multiple inheritances at the class level but can be achieved through an **interface**.

2. Member Access Rules:-

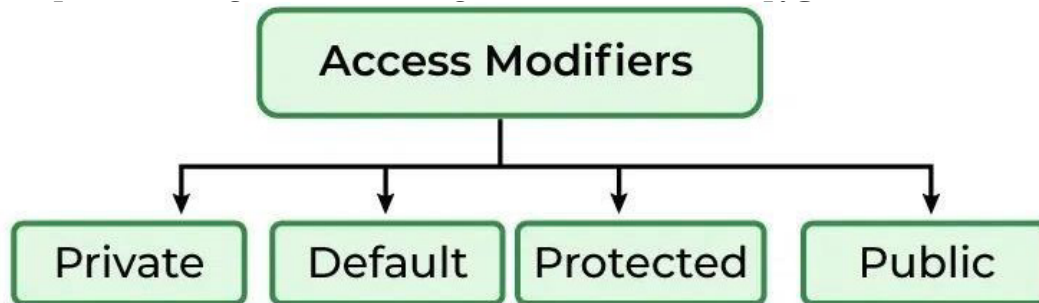
Member Access Rules in Inheritance

The following rules for inherited methods are enforced –

- Methods declared public in a superclass also must be public in all subclasses.
- Methods declared protected in a superclass must either be protected or public in subclasses; they cannot be private.
- Methods declared private are not inherited at all, so there is no rule for them.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.

We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.



	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

1) Private

The access level of a private modifier is only within the class. It cannot be accessed from outside the class and sub class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```
classA{
privateint data=40;
privatevoidmsg(){System.out.println("Hello java");
}
```

```
public class Simple
{
public static void main(String args[]){
    A obj=newA(); System.out.println(obj.data); //Compile Time Error Time Error
}
}
```

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

```
classA{
privateA(){//private constructor
void msg(){
    System.out.println("Hello java");}
}
public class Simple
{
public static void main(String args[])
{

    A obj=newA();//Compile Time Error
}
}
```

Note: A class cannot be private or protected except nested class.

2) Default

The access level of a default modifier is only within the sub classes , class and package . It cannot be accessed from outside the sub class , class and package. If you do not specify any keyword, it will be the default.

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

```
//save by A.java
```

```
package pack;
```

```
class A{  
void msg()  
{  
System.out.println("Hello");}  
}
```

```
//save by B.java
```

```
package mypack;
```

```
import pack.*;
```

```
class B{  
public static void main(String args[]){
```

```
A obj = new A();//Compile Time Error obj.msg();//Compile Time Error
```

```
}  
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) Protected

The **protected access modifier** is accessible within sub class , class and package and outside the sub class package but through inheritance only.

- Methods declared protected in a superclass must either be protected or public in subclasses; they cannot be private.
- The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
```

```
package pack;
```

```
public class A
```

```
{
```

```
protected void msg()
```

```
{
```

```
System.out.println("Hello");}
```

```
}
```

```
//save by B.java
```

```
package mypack;
```

```
import pack.*;
```

```
class B extends A
```

```
{
```

```
public static void main(String args[]){
```

```
    B obj = new B(); obj.msg();
```

```
}
```

```
}
```

Output : Hello

4) Public

The **public access modifier** is accessible everywhere. It can be accessed from within the class, outside the class, within the package and outside the package. It has the widest scope among all other modifiers.

Methods declared public in a superclass also must be public in all subclasses.

Example of public access modifier

//save by A.java

```
Package pack;  
Public class A{  
  public void msg(){  
    System.out.println("Hello");  
  }  
}
```

//save by B.java

```
package mypack;  
  
import pack.*;  
  
class B{  
  
  public static void main(String args[]){  
    A obj = new A(); obj.msg();  
  }  
}
```

3. Super Uses:-

Super Keyword in Java

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

1) super is used to refer immediate parent class instance variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

Example program:-

```
1.  class Animal{
2.    String color="white";
3.  }
4.  class Dog extends Animal{
5.    String color="black";
6.    void printColor(){
7.      System.out.println(color); //prints color of Dog class
8.      System.out.println(super.color); //prints color of Animal class
9.    }
10. }
11. class TestSuper1{
12.   public static void main(String args[]){
13.     Dog d=new Dog();
14.     d.printColor();
15.   }}
```

Output:-black white

In the above example, Animal and Dog both classes have a common property color. If we print color property, it will print the color of current class by default. To access the parent property, we need to use super keyword.

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

Example Program:-

```
1. class Animal1{
2. void eat(){System.out.println("eating...");}
3. }
4. class Dog1 extends Animal1{
5. void eat(){System.out.println("eating bread...");}
6. void bark(){System.out.println("barking...");}
7. void work(){
8. super.eat();
9. bark();
10. }
11. }
12. class TestSuper2{
13. public static void main(String args[]){
14. Dog1 d=new Dog1();
15. d.work();
16. }}
```

Output:-
eating...
barking...

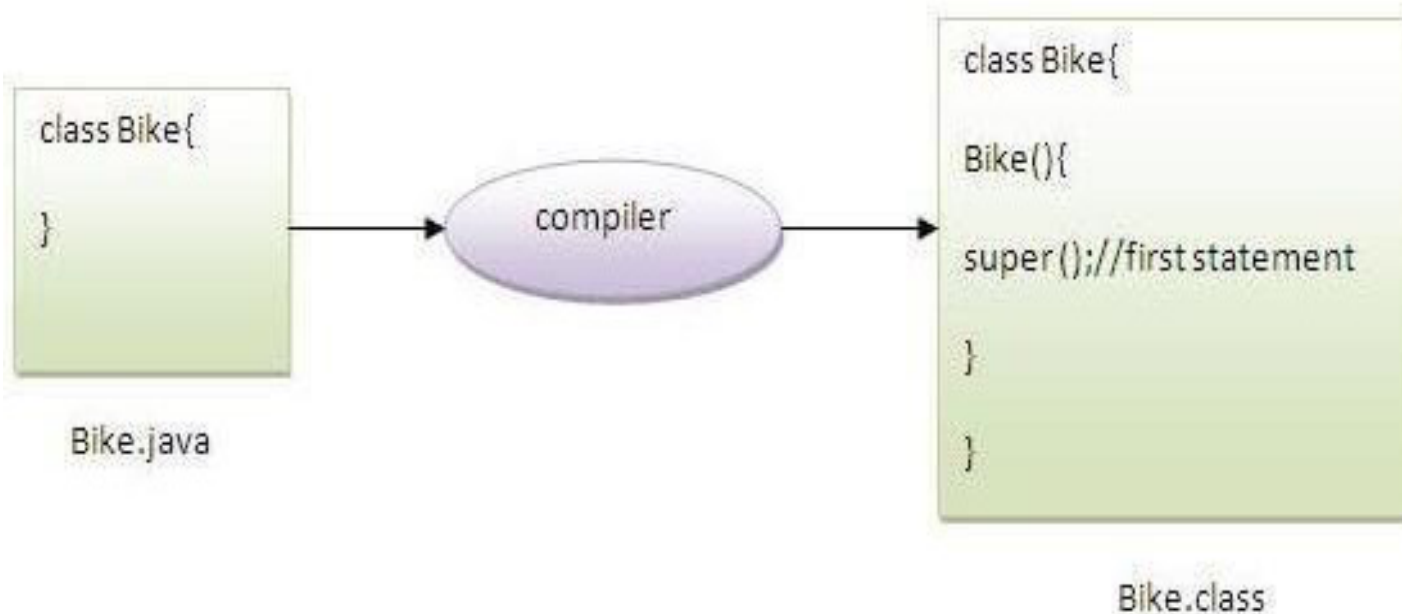
In the above example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local.

To call the parent class method, we need to use super keyword.

3) super is used to invoke parent class constructor.

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

Note: super() is added in each class constructor automatically by compiler if there is no super() or this().



Example program:-

```
1.  class Animal2{
2.  Animal2(){System.out.println("animal is created");
3.  }
4.  class Dog2 extends Animal2{
5.  Dog2(){
6.  super();//without use of super keyword java
   compiler automatically executed
7.  System.out.println("dog is created");
8.  }
9.  }
10. class TestSuper3{
11. publicstaticvoid main(String args[]){
12. Dog2 d=new Dog2();
13. }}
```

Output:-

animal is created

dog is created

Does a super class reference be used to refer sub class object?justify

Ans:-

So difference between referencing using superclass reference and referencing using subclass reference is use superclass referencing can holds object of subclass and could only access the methods which are defined/overridden by subclass while use subclass referencing can not hold

object of superclass and could access the methods of both superclass and subclass. If you

assign an object of the subclass to the reference variable of the superclass then the subclass object is converted into the type of superclass and this process is termed as widening (in terms of references).

```
class Animal{ void eat(){System.out.println("animal is eating...");} }
```

```
class Dog extends Animal{ void eat(){System.out.println("dog is eating...");}
```

```
public static void main(String    Anim  
args[]){ a=new Dog();          al  
a.eat();  
}  
}
```

4. Using Final With Inheritance:-

Final Keyword In Java

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only.

1) Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example of final variable

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

Example Program Final Variable:-

```
1.  class Bike9{
2.    finalint speedlimit=90;//final variable
3.    voidrun(){
4.      speedlimit=400;
5.    }
6.    public static voidmain(String args[]){
7.      Bike9 obj=new Bike9();
8.      obj.run();
9.    }
10. }//end of class
```

OutPut:- Compile time error //can't modify final variable value

2) Java final method

If you make any method as final, you cannot override it.

Example of final method

```
1.  class Bike{
2.      final void run(){System.out.println("running");}
3.  }
4.
5.  class Honda extends Bike{
6.      void run(){System.out.println("running safely with 100kmph");} 7.
8.      public static void main(String args[]){
9.          Honda honda= new Honda();
10.         honda.run();
11.     }
12. }
```

OutPut:- Compile time error//can't override method

3) Java final class

If you make any class as final, you cannot extend it.

Example of final class

```
1. final class Bike {}  
2.  
3. class Honda1 extends Bike {  
4. void run() { System.out.println("running safely with 100kmph"); }  
5.  
6. public static void main(String args[]) {  
7. Honda1 honda = new Honda1();  
8. honda.run();  
9. }  
10. }
```

OutPut:- Compile time error //can't inherited

5. Object class in Java

Object Class in Java:-

Object Class in Java is the topmost class among all the classes in Java. We can also say that the Object class in Java is the parent class for all the classes. It means that all the classes in Java are derived classes and their base class is the Object class. **Object** class is present in **java.lang** package.

All the classes directly or indirectly inherit from the Object class in Java.

Ex:-public class Object{

//Object class body

/*

1. toString()
2. hashCode()
3. equals(Object o)
4. clone()
5. finalize()
6. getClass()
7. wait()
8. wait(long timeout)
9. wait(long timeout,int nanos)
10. notify()
11. notifyAll()

***/**

}

/*Consider the example-

```
class Vehicle{  
//body of class Vehicle  
}
```

Here we can see that class Vehicle is not inheriting any class, but it inherits the Object class. It is an example of the direct inheritance of object class. Now, look at this example-

```
class Vehicle{  
//body of class Vehicle  
}  
class Car extends Vehicle{  
//body of class Car  
}
```

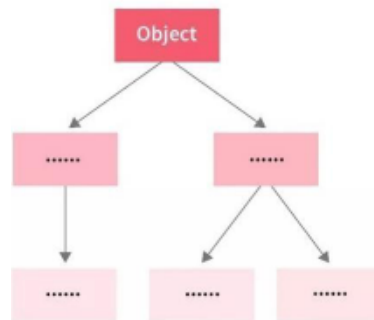


Fig.1.Object
Class

Methods of Object

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class
public int hashCode() public	can further be used to get the metadata of this the hashcode number for this object. compares class. returns
protected Object clone() throws CloneNotSupportedException boolean equals(Object obj)	the given (clone) of this object.
	object to this object. creates and returns the exact copy
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).

6.Method

Overriding:-

Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**. In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding Rules for Java Method Overriding

- 1.The method must have the same name as in the parent class
- 2.The method must have the same parameter as in the parent class.
- 3.There must be an IS-A relationship (inheritance).

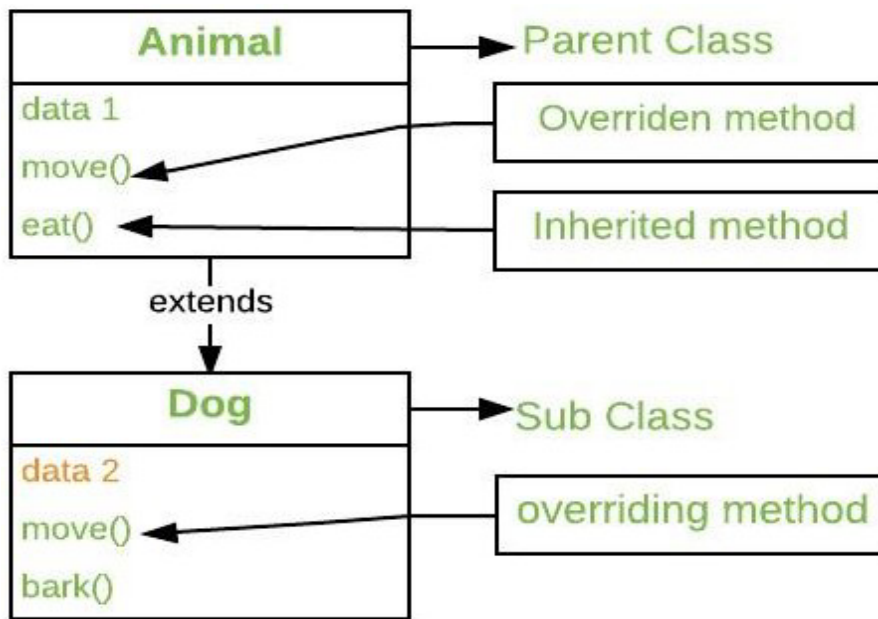


Fig.1.Method overriding process

Syntax:-class A

```
{  
Void show()  
{  
// block of code  
}  
}
```

Class B extends A

```
{  
Void show()  
{  
//block of code  
}  
}
```

Example of method overriding:-

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

//Java Program to illustrate the use of Java Method Overriding//Creating a parent class.

```
class Vehicle
```

```
{
```

```
//defining a method
```

```
void run()
```

```
{
```

```
System.out.println("Vehicle is running");}
```

```
}
```

```
//Creating a child class
```

```
class Bike2 extends Vehicle
```

```
{
```

```
//defining the same method as in the parent class
```

```
void run()
```

```
{
```

```
System.out.println("Bike is running safely");
```

```
}
```

```
public static void main(String args[]){ obj = new Bike2();//creating objectobj.run();//calling method
```

```
}
```

```
}
```

OutPut:-

**Bike is running
safely**

Static vs Dynamic Binding

Static Binding

When type of the object is determined at compiled time, it is known as static binding.

When type of the object is determined at run-time, it is known as dynamic binding.

Dynamic Binding

Understanding Type

Let's understand the type of instance.

1) **variables have a type**

Each variable has a type, it may be primitive and non-primitive.

Ex:- `int data=30;`

Here data variable is a type of int.

2) **References have a type** `class Dog{` `publicstaticvoidmain(String args[]){`

`Dog d1;`//Here d1 is a type of Dog

`}`

`}`

3) **Objects have a type**

An object is an instance of particular java class, but it is also an instance of its superclass.

```
class Animal{}
```

```
class Dog extends Animal{ public static void main(String args[]){ Dog d1=new Dog();
```

```
}
```

```
}
```

Here d1 is an instance of Dog class, but it is also an instance of Animal.

static binding

When type of the object is determined at compiled time(by the compiler), it is known as static binding.

If there is any private, final or static method in a class, there is static binding.

Dynamic binding

When type of the object is determined at run-time, it is known as dynamic binding.

Example of dynamic binding

```
classAnimal{ void eat(){System.out.println("animal is eating...");} }
```

```
class Dog extendsAnimal{ void eat(){System.out.println("dog is eating...");}
```

```
publicstaticvoidmain(String args[]){ Animal a=new Dog();
```

```
a.eat();
```

```
}
```

```
}
```

//In the above example object type cannot be determined by the compiler, because the instance of Dog is also an instance So compiler doesn't know its type, only its base type.

Output:-

Dog is eating...

8. Abstraction:-

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class:-

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Rules For Java Abstract Class:-

- o An abstract class must be declared with an abstract keyword.
- o It can have abstract and non-abstract methods.
- o It cannot be instantiated.
- o It can have [constructors](#) and static methods also.
- o o It can have final methods which will force the subclass not to change the body of the

- o To use an abstract class, you have to inherit it from sub classes
- o To use an abstract class, you have to inherit it from sub classes.o
- o If a class contain partial implementation, then we should declare a class as abstract.

Syntax:- abstract class class_name{

//abstract methods and non-abstract methods

}

Abstract Method

A method which is declared as abstract and does not have implementation is known as an abstract method.

Syntax of abstract method

abstract access modifier **return type** **method_name()**;//no method body and abstract method

Abstract class Example1:-

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike{ abstract void run();  
}  
class Honda4 extends Bike{ void run(){  
System.out.println("running safely");  
}  
public static void main(String args[]){
```

```
    Bike obj = new Honda4();  
}  
}
```

Output:- running safely

Abstract class having constructor, data member and methods

An abstract class can have a data member, abstract method, method body (non-abstract method), constructor, and even main() method.

Example 3:-

File: TestAbstraction2.java

```
1. //Example of an abstract class that has abstract and non-abstract methods
2. abstract class Bike{
3.     Bike(){System.out.println("bike is created");}
4.     abstract void run();
5.     void changeGear(){System.out.println("gear changed");} 6. }
7. //Creating a Child class which inherits Abstract class
8. class Honda extends Bike{
9.     void run(){System.out.println("running safely..");} 10. }
11. //Creating a Test class which calls abstract and non-abstract methods
14. class TestAbstraction2{.}
11. public static void main(String args[]){
14.     Bike obj = new Honda
15.     obj.run();
16.     obj.changeGear();
17. } }
```

OutPut:-

**Bike is created Running
safely..**

Gear changed

9. Abstract Method

A method declared using the **abstract** keyword within an abstract class and does not have a definition (implementation) is called an abstract method.

When we need just the method declaration in a super class, it can be achieved by declaring the methods as abstracts.

Abstract method is also called subclass responsibility as it doesn't have the implementation in the super class. Therefore a subclass must override it to provide the method definition.

Syntax for abstract method:

abstract return_type method_name([argument-list]);

Here, the abstract method doesn't have a method body. It may have zero or more.

Steps arguments to Create Abstract Method:-

- It can only be used in abstract class.
- It doesn't contain any body "{ }" and always ends with semicolon ";".
- Abstract method must be overridden in sub-classes otherwise it will also become a abstract class.
- Whenever the action is common but implementation are difficult then we should use abstract method.

Example Program:-

```
abstract class Programming
{
    abstract public void developer();
}
class C extends Programming
{
    @Override public void developer()
    {
        System.out.println("Dennis Ritchie");
    }
}
class Java extends Programming
{
    @Override public void developer()
    {
        System.out.println("James Gosling");//
    }
}
class Main
{
    public static void main(String args[])
    {
        Programming p=new C(); Java j=new Java();
        p.developer();
        j.developer();
    }
}
```

Output:-
Dennie Ritchie
James Gosling

Difference between abstract class and interface

Abstract class	Interface
1) Abstract class can have abstract and nonabstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance .	Interface supports multiple inheritance .
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables .
4) Abstract class can provide the implementation of interface .	Interface can't provide the implementation of abstract class .
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".

8) A Java **abstract class** can have class members like private, protected, etc.

Members of a Java interface are public by default.

Packages and Interfaces:-

- 1. Defining**
- 2. Creating and Accessing a Package**
- 3. Understanding CLASSPATH**
- 4. Importing Packages**

Packages:-

1.Defining:-

It is necessary in software development to create several classes and interfaces. After creating these classes and interfaces, it is better if they are divided into some groups depending on their relationship. Thus, the classes and interfaces which handle similar or same task are put into the same directory. This directory or folder is also called a package.

Defination:- A **java package** is a group of similar types of classes, interfaces and sub-packages.

There are several following advantages of package concept:

1. Java package is used to categorize the classes and interfaces so that they can be easily maintained.
2. Java package provides access protection.
3. Java package removes naming collision.

/*

- Packages are useful to arrange related classes and interfaces into a group. This makes all the classes and interfaces performing the same task to put together in the same package.
- For example, in Java, all the classes and interfaces which perform input and output operations are stored in java.io package.
- packages hide the classes and interfaces in a separate sub directory, so that accidental deletion of classes and interfaces will not take place.
- The classes and interfaces of a package are isolated from the classes and interfaces of another package. This means that we can use same names for classes of two different classes.
- For example, there is a Date class in java.util package and also there is another Date class available in java.sql package.
- A group of packages is called a library. The classes and interfaces of a package are like books in a library and can be reused several times. This reusability nature of packages makes programming easy. Just think, the packages in Java are created by JavaSoft people only once, and millions of programmers all over the world are daily by using them in various programs.

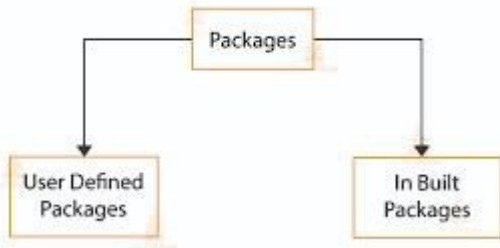
*/

Different Types of Packages

There are two different types of packages in Java. They are:

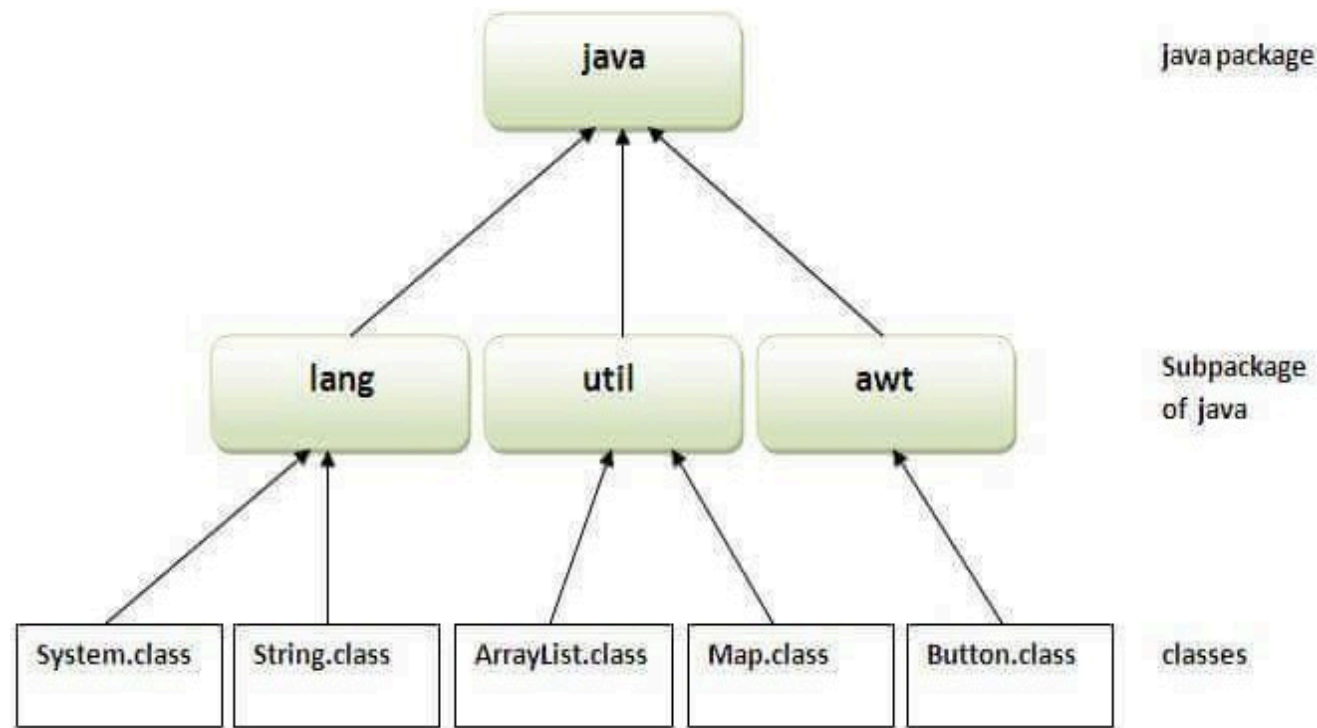
- Built-in packages
- User-defined packages

Types of Packages in Java



1. Built-in or Pre-defined Packages:-

The Package which are already created by java developer people are called pre-defined package.



Java.lang.* :-This package as primary classes and interfaces essential for Java language. It consists of Wrapper Classes . Ex: System, String, Integer, Character classes etc.,

Java.util.* :-This Package consist useful classes and interfaces like Stack, LinkedList, Vector, ArrayList, Date etc.,

java.io.* :-This packages handles Files and input, output related tasks.

Ex:- File, FileInputStream, FileOutputStream, FileReader, FileWriters.

Java.awt.* :-awt means Abstract Window Toolkit. This packages helps to develop GUI. It consists of two important some packages.

1. Java.awt.*;
2. Java.awt.event.*;
3. Java.awt.image.*;

Javax.swing.* :-This package helps to develop GUI. Java.awt will have additional features than java.awt.*;

Java.net.* :-Client and Server programming can be done using packages it uses TCP/IP TCP means Transmission Control Protocol and IP means Internet Protocol.

Java.applet.* : - Applets are small intelligent program which travel from one place to another place on Internet and executed in client side.

Java.sql.*:-This package helps to connect to the database like Oracle, MsAccess etc.,

Java.beans.*:-Beans are software reusable components in the network they can be develop this package.

Java.rmi.*:-rmi stands for Remote Method Invocation. This object which exists one computer in the network can be invoked from another computer and can be used.

Javax.servlet.* :-Servlets are Server Side Programming which handles clients.

Creating and Accessing a Package:-

User Defined Package:-

Creating a Package:-

the users of the Java language can also create their own packages. They are called user-defined packages. User defined packages can also be imported into other classes and used exactly in the same way as the Built-in packages.

Let us see how to create a package of our own and use it in any other class. To create a package the keyword **package** is used as:

Syntax:-

1. `Package packagename; //to create a package`
2. `Package packagename.subpackagename; //to create a sub package within a package`

Simple example of java package:

The **package** keyword is used to create a package in java.

```
//save as Simple.  
java package  
mypack; public  
class Simple{  
    public static void main(String[] args){  
        System.out.  
        println("Welcome to package");  
    }  
}
```

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

```
javac -d directory javafilename
```

For **example**

```
javac -d . Simple.java
```

See the output to understand how to compile a Java program that contains a package.

The `-d` option (switch) tells the Java compiler to create a separate sub directory and place the `.` class file there.

The dot(.) after `-d` indicates that the package should be created in the current directory i.e., `C:\.`. We have written as:

How to run java package program

You need to use fully qualified name e.g. `mypack.Simple` etc to run the class.

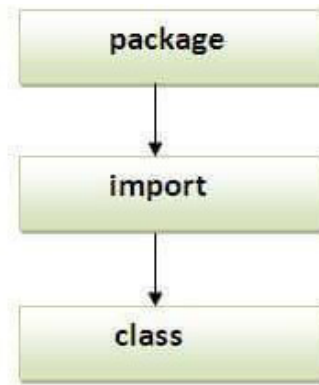
Ex:-`java mypack.Simple`

To Compile: `javac -d . Simple.java`

To Run: `java mypack.Simple`

Output:Welcome to package

Note: Sequence of the program must be package then import then class.



Note: If you import a package, subpackages will not be imported.

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Subpackage in java:-

Package inside the package is called the **subpackage**. It should be created **to categorize the package further**.

Let's take an example, Sun Microsystem has defined a package named java that contains many classes like System, String, Reader, Writer, Socket etc. These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on. So, Sun has subcategorized the java package into subpackages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

Package packagename.subpackagename; //to create a sub package within a package

Example of Subpackage

```
package high.sub;
```

```
class Simple{
```

```
    public static void main(String args[]){        System.out.println("Hello subpackage");
```

```
    }
```

```
}
```

To Compile: javac -d . Simple.java

To Run: java high.sub.Simple

Output: Hello subpackage

Accessing a Package:-

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1) Using **packagename.***

If you use **package.*** then all the classes and interfaces of this package will be accessible but not subpackages.

The **import** keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

```
//save by A.java package pack; public class A{  
    public void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java
```

```
package mypack;
```

```
import pack.*;
```

```
class B{
```

```
    public static void main(String args[])
```

```
{
```

```
    A obj = new A();
```

```
    obj.msg();
```

```
}
```

```
}
```

To Compile: `javac -d . B.java`

To Run: `java mypack.B`

Output: Hello

2) Using **packagename.classname**

If you import **package.classname** then only declared class of this package will be accessible.

Example of package by import **package.classname**

```
//save by A.java
```

```
package pack;  
public class A{  
    public void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java
```

```
package mypack;  
import pack.A;  
class B{  
    public static void main(String args[])  
    {  
        A obj = new A();  
        obj.msg();  
    }  
}
```

To Compile: javac -d . B.java

To Run: java mypack.B

Output: Hello

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

```
//save by A.java package pack; public class A{  
Example of package by import fully qualified name  
public void msg(){System.out.println("Hello");  
}  
}  
//save by B.java package  
mypack;  
class B{  
  public static void main(String args[]){  
    pack.A obj = new pack.A();//using fully  
    qualified name  
    obj.  
    msg();  
  }  
}
```

To Compile: javac -d . B.java

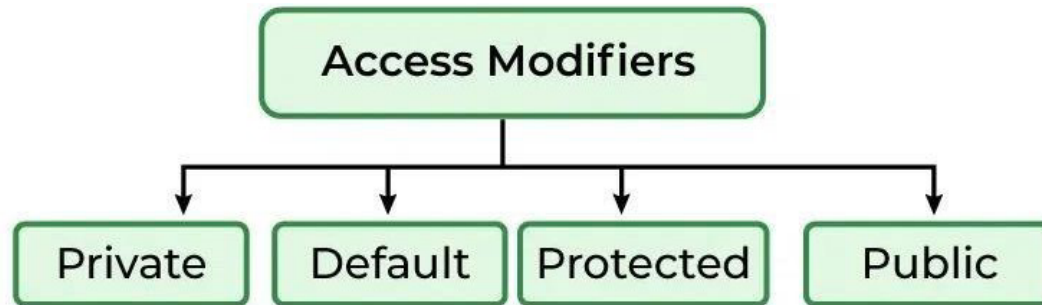
To Run: java mypack.B

Output : Hello

Accessing a Packages using Access Modifiers:-

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers using to access Packages:



1) Private

The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```

class A{
private int data=40;
private void msg(){System.out.println("Hello
java");}
}
public class Simple{
public static void main(String args[]){ A obj=new A();
System.out.println(obj.data);// Compile Time Error
obj.msg();// Compile Time Error
}
}

```

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class.

For example:

```

class A{
private A(){}//private constructor
void msg(){System.out.println("Hello java");}
}
public class Simple{
public static void main(String args[])
{
A obj=new A();//Compile Time Error
}
}

```

Note: A class cannot be private or protected except nested class.

2) Default

The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
//save by A.java
```

```
package pack; class A{  
    void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java
```

```
package mypack;  
import pack.*;  
class B{  
    public static void main(String args[]){  
        A obj = new A();//Compile Time Error obj.msg();//Compile Time Error  
    }  
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

//save by A.java

```
package pack; public class A{  
protected void msg(){System.out.println("Hello");}  
}
```

//save by B.java

```
package  
mypack;  
import pack.*;
```

```
class B extends A{  
    public static void main(String args[]){  
        B obj = new B();  
        obj.msg();  
    }  
}
```

Output:Hello

4) Public

The **public access modifier** is accessible everywhere. It can be accessed from within the class, outside the class, within the package and outside the package. It has the widest scope among all other modifiers.

Example of public access modifier

```
//save by A.java
```

```
package pack; public class A{  
public void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java  
package mypack;  
import pack.*;
```

```
class B{  
  public static void main(String  
    args[]){ A obj = new A();  
    obj.msg();  
  }  
}
```

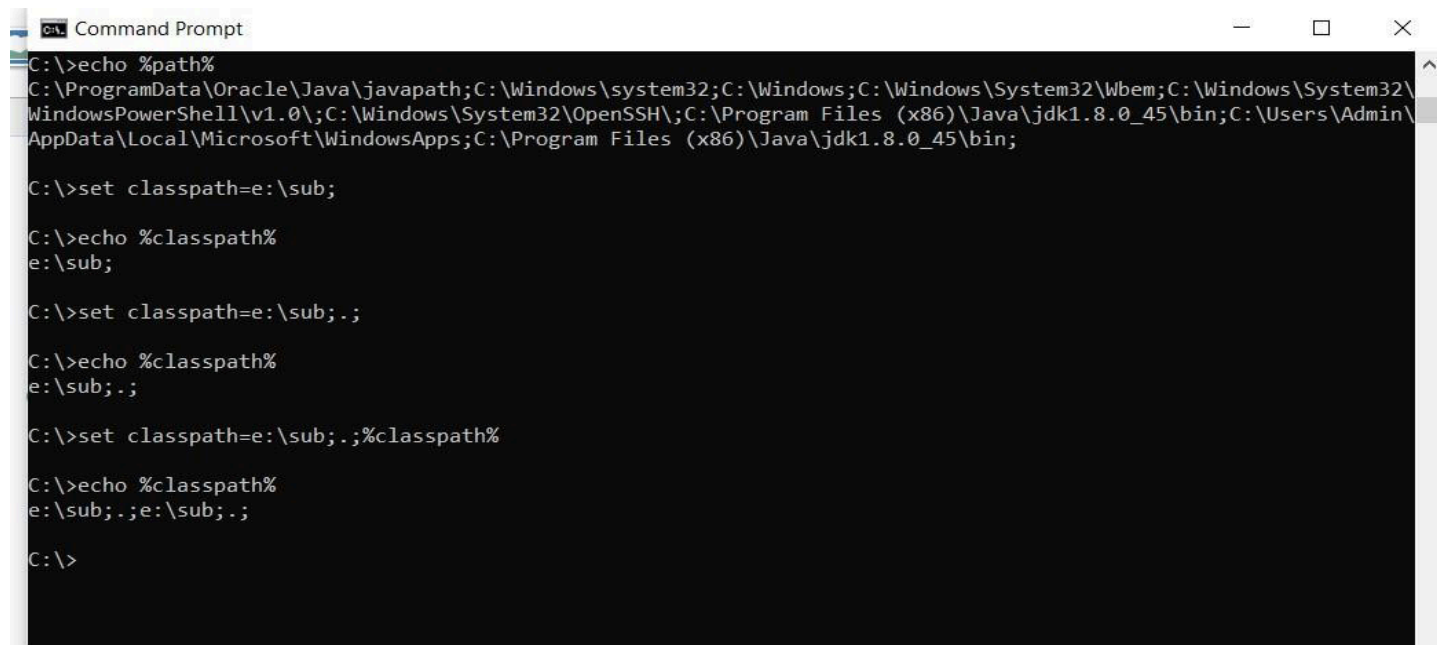
Output:Hello

Understanding CLASSPATH:-

The CLASSPATH is an environment variable that tells the Java compiler where to look for class files import. CLASSPATH is generally set to a directory or a Jar(Java Archive) file.

To see what is there currently in the CLASSPATH variable in your system, you can type in Windows

C:\>echo %path% \\used to know the all the classpaths in the particular directory



```
Command Prompt
C:\>echo %path%
C:\ProgramData\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\Java\jdk1.8.0_45\bin;C:\Users\Admin\AppData\Local\Microsoft\WindowsApps;C:\Program Files (x86)\Java\jdk1.8.0_45\bin;

C:\>set classpath=e:\sub;

C:\>echo %classpath%
e:\sub;

C:\>set classpath=e:\sub;. ;

C:\>echo %classpath%
e:\sub;. ;

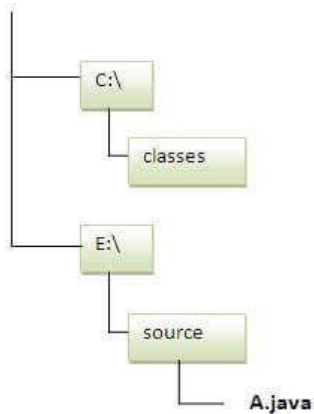
C:\>set classpath=e:\sub;. ;%classpath%

C:\>echo %classpath%
e:\sub;. ;e:\sub;. ;

C:\>
```

send the class file to another directory or drive?

There is a scenario, I am created one class A.java in a e directory folder source. I want to put the class file of A.java source file in classes folder of c: drive.



//save as Simple.

```
java package
mypack; public
class A{
public static void main(String args[]){
    System.out.println("Welcome to package");
}
}
```

To

Compile:

e:\sources> javac -d c:\classes A.java To Run:

To run this program from e:\source directory, you need to set classpath of the directory where the class

e:\sources> set classpath=c:\classes;.; e:\sources> java mypack.Simple Another

way to run this program by -classpath switch of java:

The -classpath or -cp switch can be used with javac and java tool.

To run this program from e:\source directory, you can use -classpath switch of java that tells where to look for class file. For example:

Output:Welcome to package

e:\sources> java -classpath c:\classes mypack.Simple

Ways to load the class files or jar files

There are two ways to load the class files temporary and permanent.

Temporary

- By setting the classpath in the command prompt.
- By -classpath switch

Permanent

- By setting the classpath in the environment variables
- By creating the jar file, that contains all the class files, and copying the jar file in the jre/lib/ext folder.

4. Importing Packages

There are three ways to import a package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

1) Using `packagename.*`

If you use `package.*` then all the classes and interfaces of this package will be accessible but not subpackages.

The `import` keyword is used to make the classes and interface of another package accessible to the current package.

Syntax:- `import Packagename.*;`

`import packagename.subpackagename.*;`

Example of package that import the packagename.*

```
//save by A.java package pack; public class A{  
    public void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java package  
mypack;  
import pack.*;
```

```
class B{  
    public static void main(String args[])  
    {  
        A obj = new A();  
        obj.msg();  
    }  
}
```

To Compile: javac -d . B.java

To Run: java mypack.B

Output : Hello

2) Using `package.name.classname`

If you import `package.classname` then only declared class of this package will be accessible.

Syntax:-

```
Import package.name.classname;  
import package.name.subpackage.name.classname;
```

Example of package by import `package.name.classname`

```
//save by A.java
```

```
package pack; public class A{  
    public void msg(){System.out.println("Hello");}  
}
```

```
//save by B.java
```

```
package mypack;  
import pack.A;  
class B{  
    public static void main(String args[]){  
        A obj = new A();  
        obj.msg();  
    }  
}
```

To Compile: `javac -d . B.java`

To Run: `java mypack.B`

Output : Hello

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

```
//save by A.java package pack; public class A{  
    public void msg(){System.out.println("Hello");}  
}  
//save by B.java  
package mypack;  
class B{  
    public static void main(String args[])  
    {  
        pack.A obj = new pack.A();//using fully qualified name  
    }  
}
```

To Compile: javac -d . B.java

To Run: java mypack.B

Output : Hello

1. How to put two public classes in a package?

If you want to put two public classes in a package, have two java source files containing one public class, but keep the package name same.

For example:

```
//save as A.java
package p1;

public class A{}

//save as B.java
package p1;

public class B{}
```

2. How to put same class name in a packages?

If you want to put two classes names same so, create two different package names and create same class names.

```
save as A.java

package p1;

public class A{}

//save as A.java
package p2;

public class A{}
```

The JAR Files

A Java Archive file (JAR) is a file that contains compressed version of several .class files, audio files, image files or directories.

JAR file is useful to bundle up several files related to a project and use them easily.

Let us see how to create a .jar file and related commands which help us to work with .jar files:

To create .jar file, JavaSoft people have provided jar command, which can be used in the following way:

jar cf jarfilename inputfiles

Here, cf represents create file. For example, assuming our package pack is available in C:\directory, to convert it into a jar file with the name pack.jar, we can give the command as: **C:**

\> jar cf pack.jar pack

Now, pack.jar file created.

To view the contents of a .jar file, we can use the jar command as:

jar tf jarfilename

Here, tf represents table view of file contents. For example, to view the contents of our pack.jar file, we can give the command:

C:\> jar tf pack.jar

Now, the contents of pack.jar are displayed as:

META-INF

META-INF/MANIFEST.MF

pack/ pack/Addition.class pack/Subtraction.class

The first two entries represent that there is a manifest file created and added to pack.jar file.

The third entry represents the sub directory with the name pack and the last two represents the file names in the directory pack.

When we create a .jar file, it automatically receives the default manifest file. There can be only one manifest file in an archive, and it always has the pathname

META-INF/MANIFEST.MF

This manifest file is useful to specify the information about other files which are packaged.

To extract the files from a .jar file, we can use:

jar xf jarfilename

Here, xf represents extract files from the jar file. For example, to extract the contents of our pack.jar file, we can write: C:\> jar xf pack.jar

This will create the following directories in C:\

META-INF

Pack // in this directory, we can see Addition.class and Subtraction.class

Now, we know how to create a .jar file, let us see how it can be used. In software development, any package is converted into a .jar file and stored in a separate sub directory. For example, convert our package pack into pack.jar file and store it in a sub directory e:\temp

Now set the **CLASSPATH permanently** to the pack.jar file by following the procedure shown here:

First, go to start->settings -> Control panel

In Control Panel, select System and double click on it,

System properties dialog box appears In this, select Advanced tab and then click on

Environment variables button

Go to User variables and click on New button

Set the CLASSPATH variable to pack.jar and also to the current directory, by typing at: Variable

name: CLASSPATH

Variable value : E:\temp\pack.jar;

Then click on OK button

Then click on OK in the Environment variables window and System Properties windows.

Close the Control Panel.

After setting the CLASSPATH permanently as shown in the preceding steps to the apck.jar file, it

